TestForge: Confluence Requirements → AI Test Cases → AI Automation Scripts

IT6070 Emerging Topics in IT, Assignment II · Student: I.M.R.Y. Ilangasinghe · ID: MS25914214 · Repository: [https://github.com/<user>/testforge] · Video: [link]

1. Professional, problem and evidence of need

Professional: "Nethmi" (declared pseudonym; real name and company withheld for anonymity, as promised on the interview sheet), **Senior QA Engineer**, **healthcare software industry**, at a company that builds and maintains a **Hospital Management System (HMS) for a client in Saudi Arabia**. **Role and context:** requirements are written in **Confluence** and tracked in **Jira**. A sprint has **5–6 epics** with **5–8 user stories** each. Nethmi reads the Confluence pages, writes test cases **manually**, stores them in **TestRail**, and then automates them in **Cypress**.

How it was identified: a structured 10-question interview over **WhatsApp on 28 September 2026**, using a written question sheet. **Evidence of need (from the answers):** (1) test design takes **about one working day per epic**; (2) Cypress automation takes **"normally 2 days"**; (3) under time pressure *"we do feature testing for happy path"*, so negative and boundary tests are dropped; (4) ChatGPT was tried, but *"we can't get exact test cases"*; (5) requirements are **confidential** and involve patient data; (6) to trust a tool, they want to *compare it with the story scope and see that the test cases cover all scenarios*.

Requirements: R1 (functional). From a Confluence requirements page, generate reviewable test cases for every user story and acceptance criterion, including negative and boundary cases, show coverage against the story scope, and export them to TestRail, in minutes instead of a day per epic. **R2 (functional).** From the approved test cases, generate **Cypress** automation scripts (one per story) that are checked before use, to shorten the 2-day automation effort. **R3 (non-functional).** Mask confidential and patient data (MRN, National ID/Iqama, phone numbers) before any AI call, and keep working when the AI is unavailable.

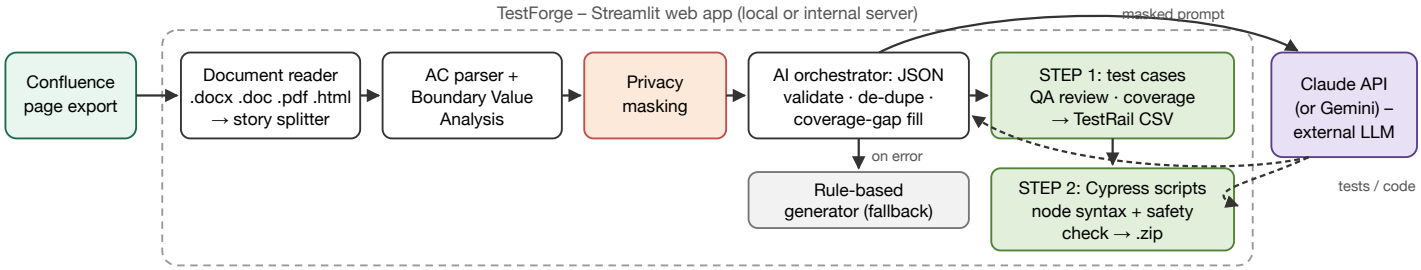
2. Emerging technology, justification and alternatives

Selected: generative AI / large language model (**Anthropic Claude**, model `claude-opus-5-5`, JSON output; Google Gemini also supported) applied to **AI-augmented software testing**. **Why it fits:** Confluence requirements are natural language. An LLM understands the intent of each story and can propose realistic positive, negative, edge and healthcare-specific scenarios (duplicate patient, invalid ID, wrong slot), write concrete steps with exact expected messages, and translate the tests into Cypress code. Nethmi's complaint that ChatGPT gave "no exact test cases" is addressed by wrapping the LLM in classic test design: **boundary values are computed exactly** and given to the model, the output must follow a schema, and **coverage of every AC is enforced and shown**.

Against requirements	LLM API: Claude (selected; Gemini supported)	Alt. A: rule/template generation (kept as fallback)	Alt. B: ChatGPT used manually (current practice)	Alt. C: self-hosted LLM (Ollama)
R1: test cases from Confluence	Reads the whole page; per-story tests; schema-validated; AC coverage enforced; TestRail CSV	Generic, mechanical tests	One story at a time by copy-paste; "no exact test cases"; no coverage check	Medium quality (8B model)
R2: Cypress scripts	Spec per story, syntax and safety checked	Skeletons only	Manual copy-paste	Possible but slower
R3: privacy and availability	Masked before sending; paid no-training tier; fallback	Fully local	Raw confidential page pasted in	Fully local (best privacy)
Cost	≈ LKR 170 per story (Opus 5.5); ≈ LKR 5 with Haiku 5.5	Free	≈ LKR 6,000 per user per month (paid plan)	GPU/16 GB+ machine; operations effort

Decision: an LLM API best meets R1 and R2 at low cost. Gemini was planned (free tier), but Google blocked API-key creation for this account ("request is suspicious"), so **Claude was integrated instead**; the provider is chosen from the key, and the design, checks and fallback are unchanged. Alternative A is embedded as the **fallback**. Alternative C is the route if the hospital client forbids external AI.

3. Architecture



4. Implemented connected features (Nethmi's workflow: Confluence → TestRail → Cypress)

Step 1: Confluence document → test cases. *Input:* a Confluence page exported as Word (.docx, or Confluence's MIME-based "Export to Word" .doc), PDF or HTML, or pasted text. *Processing:* extracts headings, lists and AC tables (PDF lines are re-joined); splits the page into user stories by story key (e.g. US-101) or "As a … I want …"; parses the criteria; computes exact boundaries (name 2–60 characters gives 1/2/3/59/60/61; ID exactly 10 digits gives 9/10/11); masks patient data; and asks Gemini for JSON test cases per story, with the page's context. It then validates, de-duplicates and **tops up any AC the AI missed**. *Output:* per-story coverage, an editable test table, a **TestRail CSV** and Gherkin files.

Step 2: test cases → automation scripts. *Input:* the test cases ticked in Step 1 (high priority or all). *Processing:* Claude writes one **Cypress** spec per story (Playwright optional), using `data-cy` selectors, `Cypress.env()` for credentials and exact expected messages. Each spec is checked with `node --check`, must contain `describe/it` blocks, and is rejected if it uses `cy.exec`, `cy.task`, `child_process` or `eval`. It is never executed. *Output:* validated `US-101.cy.js` … files in one ZIP. **Connection:** Step 2 only automates the tests the QA engineer approved in Step 1.

Error handling and fallback: a wrong file type, an empty file or one over 5 MB, a corrupt or password-protected PDF, a scanned PDF with no text, a legacy binary .doc, or a page with no stories each give a clear message and no AI call. A missing or invalid key, 429 quota error, timeout (60 s), network error, invalid JSON or zero valid items switches to the rule-based generator, with a visible warning. A partial AI answer is topped up per AC. An invalid or unsafe script is replaced by a template skeleton. Every test and script shows the engine that produced it.

5. Test cases and results

Run with `python evaluate.py`. In addition, **40 pytest unit tests** (all passing) cover all four Confluence export formats, bad documents, story/AC parsing, BVA, healthcare masking, AI JSON validation and de-duplication (mocked), every fallback path, exports, and rejection of unsafe or broken scripts. All documents are **synthetic**.

ID	Scenario	Input	Expected	Actual	Result
TC1	Confluence page → AI test cases (Claude on)	Confluence page, Word export (3 stories, 12 ACs)	Tests for all 3 stories; 100% AC coverage each; Positive, Negative and Boundary	engine = claude; 59 tests (US-101 24, US-102 20, US-103 15); coverage 100% / 100% / 100%; 160 s for 3 stories	**PASS**
TC2	PDF export read and split	Same page exported as PDF	US-101/102/103 with 5/4/3 ACs (same as Word)	US-101 (5 ACs), US-102 (4 ACs), US-103 (3 ACs)	**PASS**
TC3	**Error/fallback:** AI unavailable	Word export + invalid API key (real call to Claude)	No crash; warning shown; rule-based tests cover every AC of all stories	engine = fallback; "Claude rejected the API key"; 53 tests; coverage 100% / 100% / 100%	**PASS**
TC4	**Error:** wrong file	requirements.xlsx	Clear message; nothing sent to AI	"Unsupported file type. Use a Confluence export: .docx, .doc, .pdf or .html"	**PASS**
TC5	Test cases → Cypress scripts	High-priority tests of the 3 stories	One spec per story; each passes node syntax and safety checks	US-101.cy.js (7 it()), US-102.cy.js (8), US-103.cy.js (9), written by Claude: all syntax OK, no unsafe calls	**PASS**

6. Feasibility of adoption

Factor	Assessment
Technical	Python 3.9+, Streamlit and one HTTPS API; runs on a QA laptop or a small internal VM. Input is a standard Confluence export and output fits the existing tools (TestRail CSV, Cypress `cypress/e2e`). Node.js is optional for the syntax check.
Data / services	Pages are exported by hand today; the Confluence REST API would remove that step (needs an API token and IT approval). Needs outbound access to the Anthropic API, which hospital-project networks may restrict. The Claude API is prepaid (no free tier); a cheaper model can be set with `CLAUDE_MODEL`.
Skills / time	The QA team already knows TestRail and Cypress. About 2 h of training on reviewing AI output. About 1 week to integrate (SSO, TestRail template, Cypress `data-cy` naming). About 4 h a month for upkeep.
Privacy / security	Confidential healthcare requirements, subject to Saudi Arabia's PDPL, the client contract, and Sri Lanka PDPA No. 9 of 2022 if the team is in Sri Lanka. Mitigations: MRN, ID, phone and e-mail masking; synthetic data only; paid no-training tier; keys in .env or memory; generated code checked and never executed automatically. Residual risk: business rules in the page are still sent, so client approval or a self-hosted model is needed.

Benefits: a whole Confluence page goes to test cases in minutes; negative and boundary tests that are skipped today come back; visible AC coverage; TestRail-ready output; Cypress specs as a starting point. **Limitations:** the LLM can miss clinical or business rules or invent behaviour. Unusual page layouts may not split into stories. Scanned PDFs cannot be read. Patient names are not detected by masking. Specs are skeletons whose selectors must be confirmed, and they are not run in the PoC. **Risks and mitigations:** over-trust in a patient-safety domain (mandatory review; engine and coverage labels); data leakage (masking, paid tier, local model); unsafe code (syntax and safety gate); model or price change (configurable model, fallback). **Improvements:** Confluence and TestRail API sync, RAG over the HMS's past test cases, and running specs against a staging URL.

7. Cost estimate and affordability (LKR, 1 USD ≈ LKR 300)

Assumptions (workload from the interview): about 5.5 epics per sprint with two-week sprints gives about 11 epics (≈ 70 stories) a month. Test design takes 1 day (8 h) per epic today. Each story needs about 1.5 AI design runs and one script. **Measured** with Claude Opus 5.5 on the largest sample story: ≈ 1.1k input + 8.5k output tokens per design run and ≈ 3.4k input + 14k output tokens per script (output includes the model's reasoning). At the list price of USD 4 / 20 per million input / output tokens (verify current pricing), that is about USD 0.56 (≈ LKR 170) per story. QA time is costed at LKR 2,500 per hour. Existing laptops are reused, and the team is assumed to be based in Sri Lanka.

Setup (one-off)	LKR
Hardware (existing laptops / VM)	0
Hardening, SSO, TestRail and Cypress templates (40 h × 3,000)	120,000
QA team training (2 h + preparation)	35,000
Licences (Streamlit, Cypress: open source)	0
Total setup	**155,000**

Recurring (per month)	LKR
Claude API, Opus 5.5 (≈ 70 stories × LKR 170)	≈ 12,000
Internal VM hosting (2 vCPU / 2 GB)	≈ 6,000
Maintenance: prompts, rules, model updates (4 h)	12,000
Total recurring	**≈ 30,000**
PoC as built (laptop, synthetic documents; test runs ≈ USD 2)	*≈ 600*

Expected value: if designing one epic's tests drops from 8 h to about 3 h (reviewing instead of writing), that saves 5 h × 11 epics ≈ **55 h a month ≈ LKR 137,500**, about **4.5× the recurring cost**. Setup pays back in about 1.4 months. Switching to Claude Haiku 5.5 would cut the API line from ≈ LKR 12,000 to ≈ LKR 300 a month, if review shows its tests are good enough. This excludes Cypress time saved from the 2-day automation effort. **Lower-cost alternative:** rule-based templates only (LKR 0 recurring, but generic tests). **Current practice:** ChatGPT used manually costs about LKR 6,000 per user per month on a paid plan, but gives no exact tests, no coverage and a privacy risk. **Judgement: affordable.** The main costs are one-off integration and upkeep. Only synthetic data was used in the PoC.

8. Adoption / commercialisation approach

Path: (1) a two-sprint pilot on 2–3 HMS epics in Nethmi's team, comparing design and automation hours, the number of negative and boundary tests, and defects found against manual epics; (2) an internal rollout with SSO, TestRail import and Cypress conventions, and later a Confluence API connector; (3) optionally, a Confluence/TestRail add-on for QA teams at Sri Lankan software houses, priced per QA seat. **Who pays:** the company's QA/engineering budget. The saved QA days improve margins on the fixed-scope HMS contract. **Main condition for proceeding:** the healthcare client and company security must approve sending *masked* requirements to an external LLM under a data-processing agreement (otherwise a self-hosted model is needed), **and** the pilot must show at least 30% less design time, with at least 80% of AI tests accepted after review.

AI Use Declaration

AI tools were used as the brief encourages, and the student remains responsible for correctness. **Claude, called through the Anthropic API,** is also part of the *product* (the emerging technology being evaluated); that use is product behaviour, not authorship of the submission. **The interview answers were given by the real professional and were not generated by AI.** Add a row for any other AI tool you used yourself.

AI tool	Purpose	Prompt or instruction (summarised)	Output used	How the output was verified or corrected
Claude (Anthropic) in Claude Code	Topic selection and ideas	"This is my second assignment of emerging technology … please select relevant topics … do it properly with guidelines." Chosen: software quality assurance.	Problem idea, technology comparison	Checked against every task item and rubric row. **Validated with a real QA engineer**, whose answers changed the design (below).
Claude	Interview preparation	Prepare questions to identify a QA engineer's test-design problems	10-question interview sheet	Sent to the engineer on WhatsApp (28 Sep 2026); the answers were written by the engineer. No answers are AI-generated.
Claude	Adapting to evidence and feedback	Update the solution to the interview answers; then "once I give the Confluence document I want to create test cases, and next generate automation scripts – that's enough"	Confluence document input (docx/doc/pdf/html) with story splitting; Cypress output; healthcare samples and masking; simplified two-step UI; requirement-quality feature removed	Each change traced to an interview answer (Q2 Confluence, Q3, Q7 Cypress, Q9, Q10) or my own scope decision. New unit tests for all four export formats, bad documents and Cypress checks.
Claude	Code generation	Implement the modules and Streamlit UI (Python 3.9)	`testforge/`, `app.py`	Run and inspected. **Defects found and fixed:** PDF exports broke sentences across lines, giving 8 ACs instead of 5 (re-flow added); a prompt-building operator-precedence bug that dropped the system prompt (caught by unit tests); missed "maximum … is N" and "exactly N digits" boundaries; an over-strict code check; noisy labels.
Claude	Prompt engineering	Prompts for strict-JSON test cases using BVA hints and page context, and for safe Cypress/Playwright code	`TEST_DESIGN_PROMPT`, `CYPRESS_PROMPT`, `SCRIPT_PROMPT`	Output is never trusted directly: schema validation, de-duplication, coverage top-up and syntax/safety gates are unit-tested. Real Claude output for TC1 was compared against the ACs (100% coverage; positive, negative and boundary tests).
Claude	Test data and testing	Synthetic Confluence page in 4 export formats; pytest suite; UI walkthrough	sample_docs/, 40 unit tests, 5 formal cases	40/40 pass; TC2–TC5 pass; TC3 uses a real invalid-key call; both steps exercised in a browser. No real patient or client data was used.
Claude	Documentation	Draft the README, this document, the video script, the weekly forms and the progress slides	README, this PDF, video script, forms, PPTX	Cost arithmetic recomputed by hand; prices and legal references to be checked against official sources. Reworded in my own words.

Responsibility statement: I have reviewed, run and tested all AI-generated material, and I can explain every part of the submitted solution. The interviewee's consent to use anonymised answers is kept (WhatsApp message).